MLOPS ASSIGNMENT 01

End-to-End Heart Disease Classification System

Experiment Tracking • Automated Testing • Containerization • Kubernetes • Monitoring

Machine Learning Operations (MLOps)

Course: AIMLCZG523 | Assignment 01

Student name: Amlan Gopal Dhalasamanta

Student ID: 2025ae05188

Repository: https://github.com/2025ae05188/heart-disease-mlops

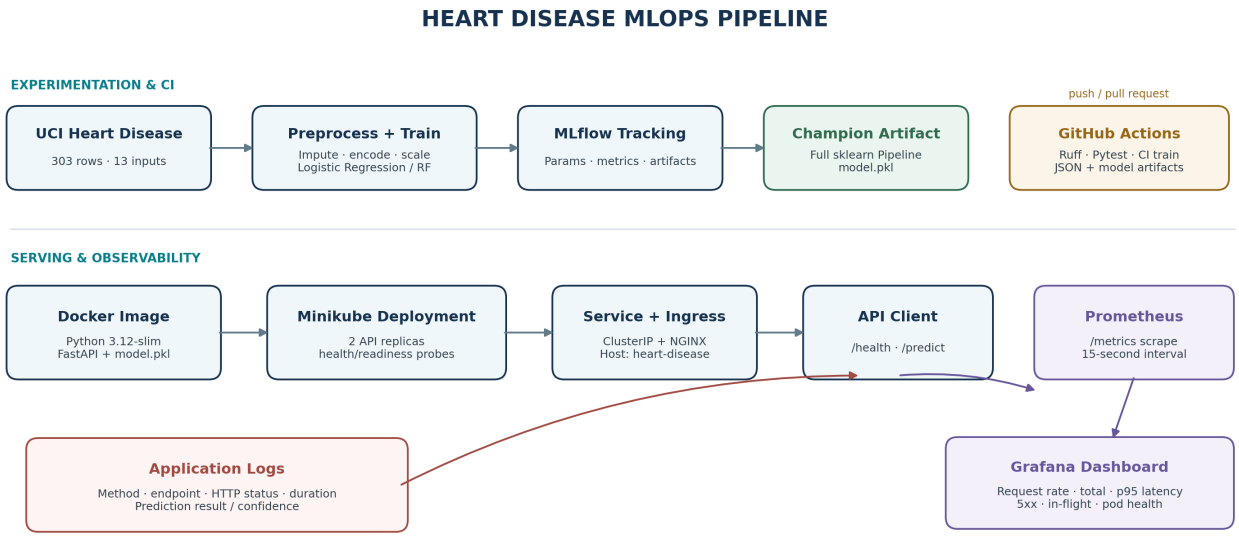
Submission date: 14/10/2026

Executive Summary

This project implements an end-to-end MLOps workflow for classifying the presence or absence of heart disease using the UCI Heart Disease dataset. The workflow covers data acquisition and exploratory analysis, reproducible preprocessing, comparison and tuning of classification models, MLflow experiment tracking, model packaging, automated checks and CI training, Docker-based API serving, local Kubernetes deployment, and request monitoring with Prometheus and Grafana.

The selected candidate is a Logistic Regression pipeline with C=2.0. These are results from this dataset and split, not evidence of clinical validity.

System Architecture



Local Kubernetes endpoint and monitoring UI are accessed through the VM; public internet access was not verified.

Implementation Snapshot

Setup and instruction commands to run: Refer Runbook.md

Area	Implementation
Data / model	UCI Heart Disease (303 records); Logistic Regression and Random Forest; stratified 80/20 split; 5-fold stratified CV.
Experiment tracking	MLflow with SQLite tracking store; experiment heart-disease-classification; registered model heart-disease-classifier.
Automation	GitHub Actions: Ruff, Pytest, fresh CI model training, and workflow artifact upload.
Serving / deployment	FastAPI + scikit-learn Pipeline in Docker; two replicas on Minikube; ClusterIP Service and NGINX Ingress.
Monitoring	Instrumented API metrics/logs; Prometheus scrapes ready API Pods; Grafana dashboard with six panels.

1. Data Acquisition and Exploratory Data Analysis

The project uses the UCI Heart Disease dataset accessed through the ucimlrepo package (dataset ID 45). The original target column is num; it is converted to a binary label by mapping num=0 to class 0 and num>0 to class 1. This aligns the training objective with the assignment requirement to classify presence versus absence of heart disease.

Dataset property	Recorded result
Observations / predictors	303 rows and 13 input features; target column num.
Class distribution	164 class 0 (54.13%); 139 class 1 (45.87%).
Missing values	ca: 4 values (1.32%); thal: 2 values (0.66%). Missing markers such as ? are converted to missing values.
Duplicates	No duplicate rows found in the initial data audit.
Numerical features	age, trestbps, chol, thalach, oldpeak.
Categorical/discrete features	sex, cp, fbs, restecg, exang, slope, ca, thal; ca is treated as categorical.

Cleaning and EDA Decisions

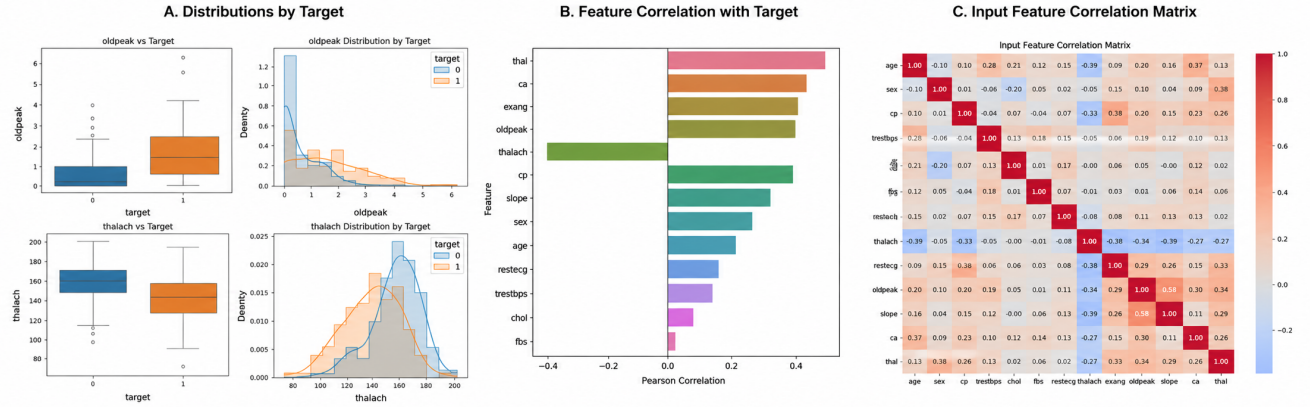
- Use a reproducible download script rather than a manually maintained copy of the source data.
- Inspect target balance, missingness, numerical distributions, category frequencies, and feature relationships before modelling.
- Impute numerical missing values with the median; impute categorical missing values with the most frequent value.
- Use one-hot encoding with handle_unknown="ignore" for categorical columns. Avoid removing potential outliers automatically without domain justification.
- Initial correlation screening showed associations for thal (~+0.522), ca (+0.460), exang (+0.432), oldpeak (+0.425), thalach (-0.417), and cp (+0.414); encoded categorical correlations require cautious interpretation.

Relevant Implementation Areas

Data acquisition and preparation are implemented in src/heart_disease_ml/data/ and src/heart_disease_ml/preprocessing/, with scripts/download_data.py supporting dataset acquisition. The final evidence folder should include the class-balance plot, key distributions, missing-value summary, and correlation visualization generated by the project.

> **SECTION SUMMARY** The source dataset is reproducibly acquired, the binary target is defined explicitly, and missingness and feature types are documented before model training.

EVIDENCE PATH *evidences/sections 1-2*



2. Feature Engineering and Model Development

Two classification pipelines were developed and compared: Logistic Regression and Random Forest. A stratified 80/20 train/test split is used with random_state=42. Transformations are fit only on training data through scikit-learn Pipelines and ColumnTransformers, reducing leakage from the held-out test set. Five-fold StratifiedKFold cross-validation is applied to the training split.

Design choice	Implementation and rationale
Split and validation	80/20 stratified train/test split; 5-fold stratified CV on training data.
Logistic Regression	Numeric median imputation and scaling; categorical most-frequent imputation and one-hot encoding; max_iter=1000.
Random Forest	Same train/test and categorical-preprocessing principles; numeric scaling is not required for tree splits.
Evaluation	Accuracy, precision, recall, F1 and ROC-AUC; ROC-AUC used as the selection metric.
Excluded transformations	No SMOTE, PCA, automatic outlier removal, or feature selection was used in the baseline phase.

Recorded Tuning Setup for section 3

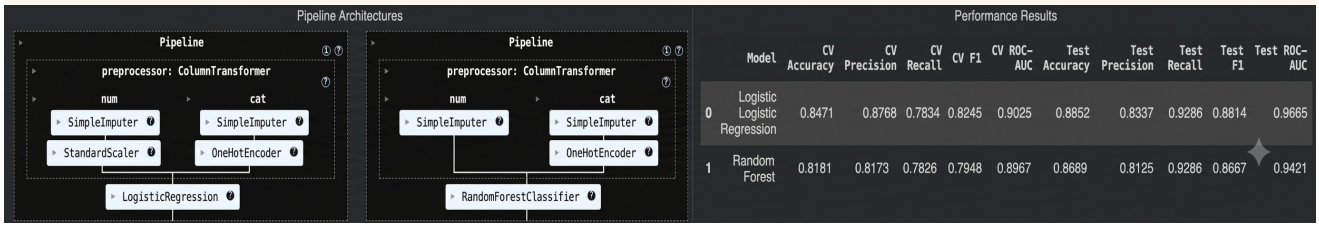
- Logistic Regression C grid: [0.05, 0.5, 2.0, 20.0].
- Random Forest grid: n_estimators [150, 250], max_depth [4, 8, 12], min_samples_split [3, 6], min_samples_leaf [1, 3].
- The tracked search included 32 entries: two baseline runs, two parent tuning runs, four Logistic Regression candidates and 24 Random Forest candidates.

Baseline Results

Metric	Recorded value
Selected model	Logistic Regression, C=2.0
Mean CV ROC-AUC	0.9025
Holdout accuracy	0.8852
Holdout precision / recall	0.8387 / 0.9286
Holdout F1 / ROC-AUC	0.8814 / 0.9665

SECTION SUMMARY Logistic Regression was selected as better one as compared to random fore, with CV ROC-AUC 0.9025 and holdout ROC-AUC 0.9665 on the recorded split.

EVIDENCE PATH *evidences/sections 1-2*



3. Experiment Tracking with MLflow

MLflow is used to make model experiments inspectable and comparable. The tracking URI is sqlite:///mlflow.db and the experiment is heart-disease-classification. Baseline experiments and tuning candidates log parameters, metrics, artifacts, evaluation output and plots. Tuning candidates are nested under parent runs, and tags record the model family, assignment section, tuning method, dataset identifier/fingerprint, preprocessing and selection metric.

MLflow item	Recorded configuration
Experiment	heart-disease-classification
Tracking store	SQLite: sqlite:///mlflow.db (local working store)
Registered model	heart-disease-classifier
Champion alias	champion -> model version 1
Best candidate run	fdc57e6f11bb430d83e6b29455b2e958
Parent tuning run	f21f10428aab407eaf75178d17e799b8

The registered candidate is inspectable in the MLflow UI under the source run artifacts, including the model artifact. MLflow is part of the experimentation and registry workflow; it is not a startup dependency of the deployed API. A fresh MLflow run is required for the live demonstration, while the larger tuning search need not be repeated.

SECTION SUMMARY MLflow records the experiment history and selected registry version; local SQLite and run artifacts must be preserved separately if interactive history is needed after VM shutdown.

EVIDENCE PATH *evidences/sections 3-4*

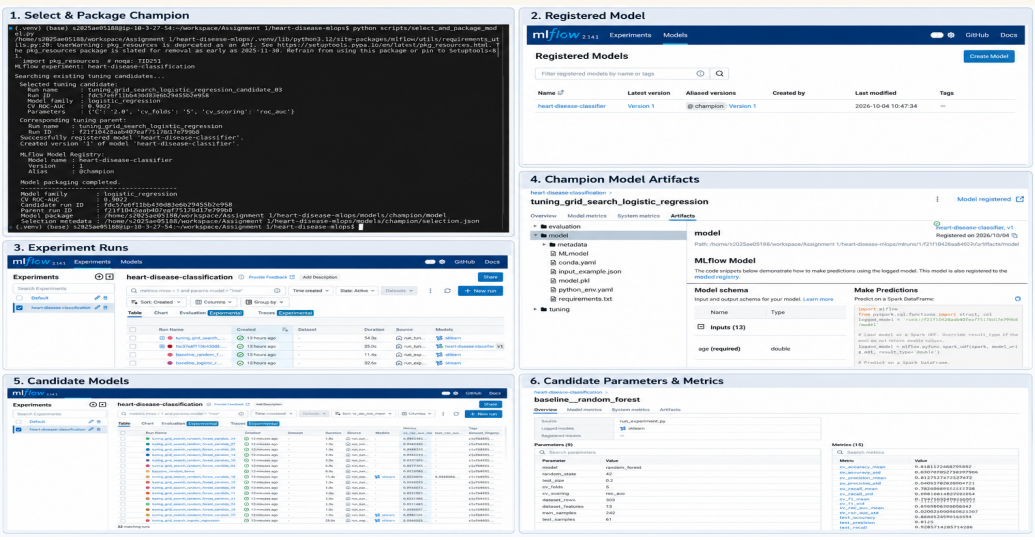
4. Model Packaging and Reproducibility

The selected object is the full fitted scikit-learn Pipeline, including preprocessing and classifier stages, exported as models/champion/model.pkl. Saving the entire pipeline keeps imputation, encoding and scaling consistent between training and inference. The pickle is tracked in Git so a clean clone contains the artifact required by the serving Docker build. The requirements.txt will be used in subsequent sections based on the requirements with the same model.

Artifact / configuration	Purpose
models/champion/model.pkl	Standalone serving artifact; 5,207 bytes in the committed snapshot.
requirements.txt	Core model/data dependencies with pinned versions.
requirements-mlflow.txt	Core dependencies plus MLflow 2.14.1 and SQLAlchemy 2.0.31.
requirements-dev.txt	Pytest, pytest-cov and Ruff for quality checks.
requirements-serving.txt	Pinned FastAPI, Uvicorn, scikit-learn, NumPy, pandas and joblib requirements.
requirements-monitoring.txt	Includes serving dependencies and prometheus-client==0.20.0.

SECTION SUMMARY The deployed model is a reusable preprocessing-plus-classifier pipeline, versioned as a small pickle artifact and loaded without MLflow at serving time.

EVIDENCE PATH *evidences/sections 3-4*



5. CI/CD Pipeline and Automated Testing

The GitHub Actions workflow provides automated quality checks and a fresh model-training validation for repository changes. It checks out the repository, configures Python 3.12, installs development dependencies and the package, runs Ruff, downloads the dataset, runs Pytest, trains a new Logistic Regression model for CI, and uploads outputs as workflow artifacts.

Workflow Stages

Stage	Expected output / check
Checkout + Python setup	Source checkout; Python 3.12 runtime.
Dependency installation	requirements-dev.txt and editable package install.
Lint	ruff check . must pass.
Data acquisition	Download the UCI dataset in the runner.
Unit tests	8 tests passed in the most recently recorded local verification.
Fresh CI training	scripts/ci_train.py trains Logistic Regression with C=2.0, evaluates metrics and saves a CI model.
Artifact upload	ci-training-artifacts contains artifacts/ci/ci_model.pkl and artifacts/ci/metrics.json.

Failure Behavior and Artifact Separation

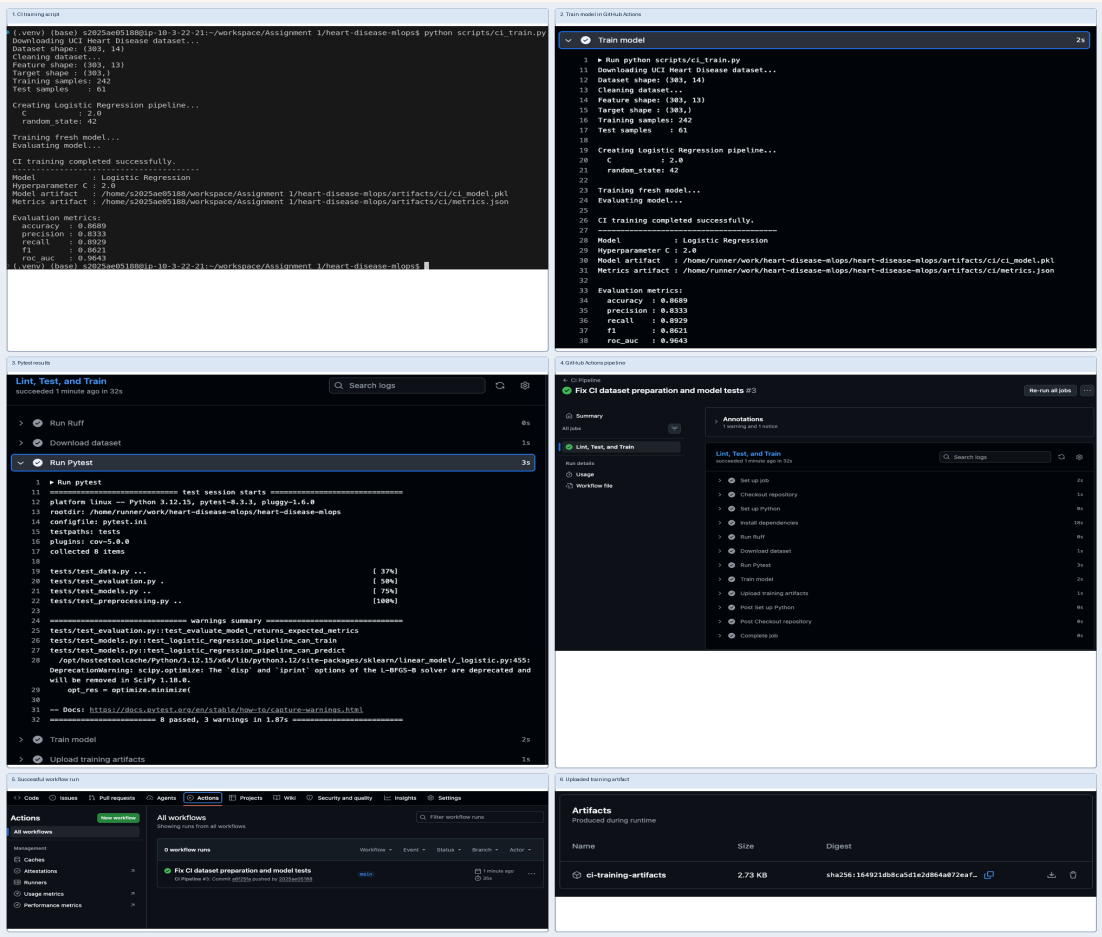
The workflow should fail when lint, test, dataset download, or training validation fails. CI model output is an evaluation artifact; it does not overwrite or promote models/champion/model.pkl. The previously observed workflow passed, with Ruff and eight tests successful, and the artifact bundle was available from the Actions run. For the live demonstration, capture the newly triggered run created by a harmless comment-only commit and record its actual commit/run link after it completes.

Repository

https://github.com/2025ae05188/heart-disease-mlops

SECTION SUMMARY GitHub Actions links code quality, tests and a fresh CI training run to a commit, and publishes the CI model plus metrics without changing the serving champion.

EVIDENCE PATH *evidences/section 5*



6. Model Containerization and FastAPI Serving

The model is served by FastAPI inside a Docker image based on python:3.12-slim. The API loads models/champion/model.pkl through joblib and reuses the fitted Pipeline. No MLflow server or tracking database is needed at runtime. The Dockerfile is kept at src/Dockerfile, with its associated ignore file at src/Dockerfile.dockerignore; build commands are run from the repository root.

API Contract

Endpoint	Purpose
GET /health	Readiness-style health response; returns status and whether the model is loaded.
POST /predict	Accepts JSON with the 13 expected feature fields; responds with class prediction, label, confidence and class probabilities.
GET /metrics	Exposes Prometheus metric samples; scrapes are excluded from application request totals.

Feature Order

age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal

Example Prediction Request

```
curl -i -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,
       "fbs":1,"restecg":2,"thalach":150,"exang":0,
       "oldpeak":2.3,"slope":3,"ca":0,"thal":6}'
```

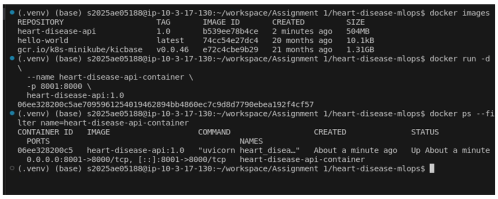
The recorded response for this sample was HTTP 200, prediction 0, prediction_label no_heart_disease_detected, confidence approximately 0.8805, and probabilities of 0.8805 for no_heart_disease and 0.1195 for heart_disease_present. Confidence is a model probability, not a clinical diagnosis.

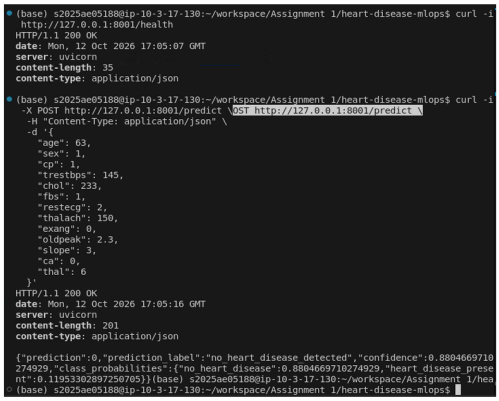
Build

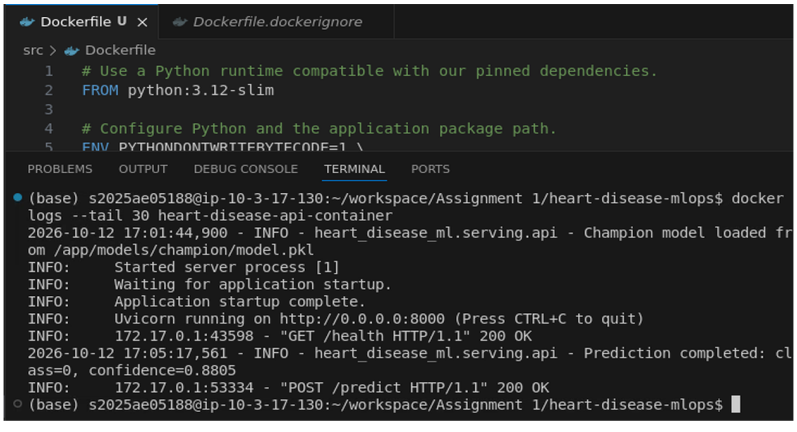
```
docker build -f src/Dockerfile -t heart-disease-api:1.1 .
```

SECTION SUMMARY A local Docker image packages the API, pinned runtime dependencies and the full champion pipeline; /health, /predict and /metrics were tested successfully.

EVIDENCE PATH *evidences/section 6*







7. Production Deployment on Local Kubernetes

The API is deployed to Minikube, which is an allowed local Kubernetes option in the assignment. The committed manifests define a two-replica Deployment, a ClusterIP Service, and an NGINX Ingress. The image heart-disease-api:1.1 is loaded into Minikube and uses imagePullPolicy: Never so the cluster uses the locally loaded image. Readiness and liveness probes call /health. Config files are also got added on top of this to support the monitoring with Prometheus and Grafana as part of next sections.

Manifest	Role
k8s/deployment.yaml	Two replicas, container port 8000, health probes, CPU/memory requests and limits.
k8s/service.yaml	Service port 80 routes traffic to container port 8000.
k8s/ingress.yaml	NGINX Ingress routes HTTP requests with Host: heart-disease to the Service.
k8s/monitoring/prometheus.yaml	Prometheus Deployment, Service, scrape configuration and restricted Pod-discovery RBAC.
k8s/monitoring/grafana.yaml	Grafana Deployment/Service and ConfigMap mounts; admin credentials supplied through a Kubernetes Secret.
k8s/monitoring/grafana-config.yaml	Prometheus datasource, dashboard provider and six-panel dashboard JSON.

Verification Commands

```
minikube status
minikube kubectl -- get pods -o wide
minikube kubectl -- get svc heart-disease-api
minikube kubectl -- get ingress heart-disease-api
minikube kubectl -- rollout status deployment/heart-disease-api
```

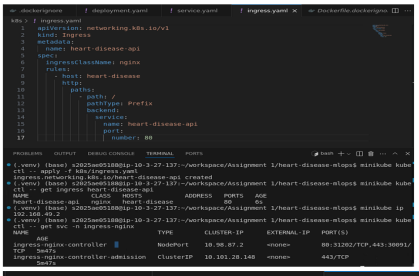
Ingress Test

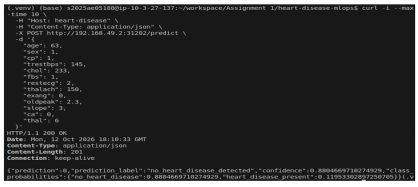
```
MINIKUBE_IP=$(minikube ip)
curl -i --max-time 10 -H "Host: heart-disease" "http://${MINIKUBE_IP}/health"
```

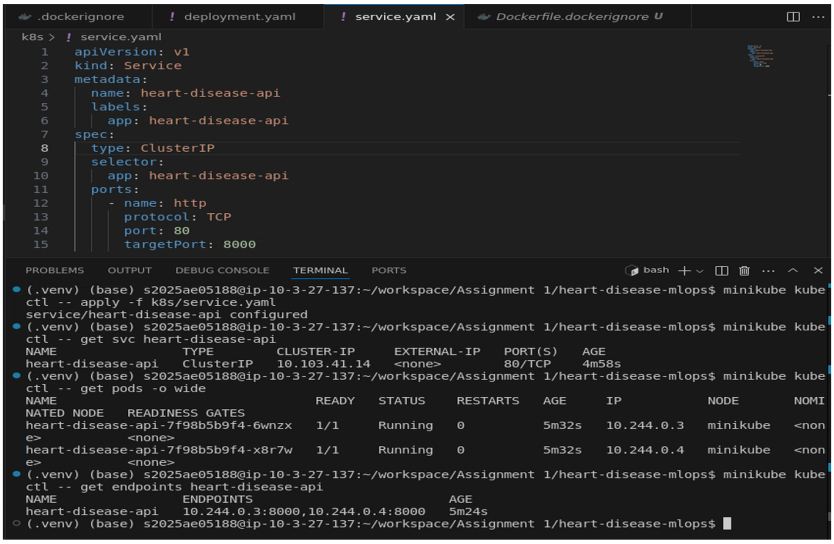
The Ingress host heart-disease is a local routing host, not a globally registered DNS name. Health and prediction routes were verified through Minikube Ingress. Public inbound routing and public DNS were not verified; therefore the submission should provide local access instructions rather than claim a public API URL.

SECTION SUMMARY The API is running behind a Kubernetes Service and NGINX Ingress on Minikube, with two replicas and health probes; the endpoint is locally verified, not publicly hosted.

EVIDENCE PATH *evidences/section 7*







8. Monitoring and Logging with Prometheus and Grafana

The API emits structured request logs and Prometheus-compatible metrics through middleware. Each request records the method, normalized endpoint, HTTP status code and duration; prediction handling also logs the result/confidence. The middleware tracks in-progress requests and excludes Prometheus scrapes of /metrics from application traffic totals. Patient input values are not used as metric labels.

API Metrics

Metric	Type	Purpose
heart_disease_api_requests_total{method, endpoint, status_code}	Counter	Request totals grouped by method, endpoint and status code.
heart_disease_api_request_duration_seconds	Histogram	Request duration distribution; supports percentile latency calculations.
heart_disease_api_requests_in_progress	Gauge	Number of requests currently being handled.

Prometheus Discovery and Grafana

- Prometheus uses image prom/prometheus:v2.55.0 and scrapes every 15 seconds.
- Kubernetes service-account/RBAC allows discovery of ready, running Pods labelled app=heart-disease-api and the named http port.
- The verified query up{job="heart-disease-api"} returned two API target series with value 1 when tested.
- Grafana uses image grafana/grafana:11.2.2 and the in-cluster datasource http://prometheus.default.svc.cluster.local:9090.
- Dashboard: Heart Disease API Monitoring (folder MLOps), with panels for request rate, total requests, p95 latency, HTTP 5xx errors, in-progress requests, and API Pod scrape health.

Useful Queries

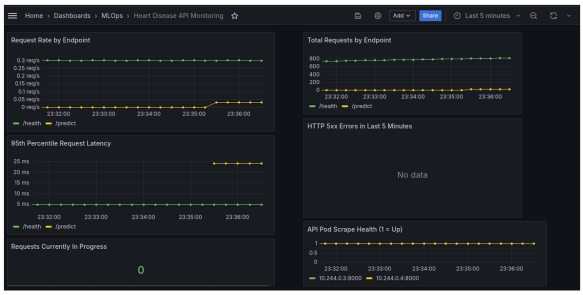
```
up{job="heart-disease-api"}
sum by (endpoint) (heart_disease_api_requests_total)
sum(rate(heart_disease_api_requests_total[5m])) by (endpoint)
```

Prometheus and Grafana are cluster services exposed for local review through port-forward commands. Dashboard values vary with actual traffic and should be captured live, not reported as fixed service guarantees. Grafana credentials are stored in a Kubernetes Secret and must not appear in screenshots or source control.

> **SECTION SUMMARY** Prometheus scraped both API replicas and Grafana displayed six operational panels; the monitoring stack supports visibility into traffic, latency, failures, in-flight requests and pod health.

EVIDENCE PATH *evidences/section 8 and final recording video: evidences/recording*







9. Documentation, Reproducibility and Submission Notes

This report consolidates the implementation across Sections 1-8 and maps the required evidence to the repository. The source of truth for code and manifests is the GitHub project. The final submission should include this report, a curated screenshot folder, local API access instructions, and the short overall-pipeline video required by the assignment.

Clean Setup and Reproduction

```
git clone https://github.com/2025ae05188/heart-disease-mlops.git
cd heart-disease-mlops
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -r requirements-mlflow.txt
python -m pip install -r requirements-dev.txt
python -m pip install -e .
python scripts/download_data.py
```

Use the project's documented script/configuration for each task. MLflow records are stored locally in mlflow.db and its associated artifact/run directory; they are not restored by downloading a GitHub ZIP. Start the MLflow UI against the same tracking URI after the experiment has been run. For serving, build the Docker image from the repository root and follow the Kubernetes/monitoring manifests and the accompanying demo runbook.

Local Access Instructions

- API health: curl with the Host: heart-disease header against the current Minikube IP and Ingress port; confirm HTTP 200 and model_loaded=true.
- API prediction: POST JSON containing all 13 expected feature names to /predict; retain the returned prediction, label, confidence and class probabilities as demonstration evidence.
- Prometheus: forward Service prometheus to localhost:9090 and inspect /targets or query up{job="heart-disease-api"}.
- Grafana: forward Service grafana to localhost:3000 and open the provisioned Heart Disease API Monitoring dashboard. Supply the Secret-backed login privately.

Limitations and Responsible Interpretation

The dataset contains 303 records and the metrics reflect a single recorded split and validation process. Performance is not a substitute for clinical validation, prospective testing, calibration review or regulatory assessment. The deployment is a local Minikube deployment; public DNS and public inbound access have not been established. MLflow history and monitoring time series use local/ephemeral storage in this environment and should be exported or captured when preservation is required.

Final Section Summaries

Section	Summary
1. Data / EDA	UCI dataset acquired; target binarized; missingness and feature groups documented.
2. Modelling	LR and RF pipelines compared; selected LR C=2.0 from recorded CV/tuning results.
3. MLflow	Experiment runs, parameters, metrics, artifacts and champion registry metadata tracked.
4. Packaging	Full scikit-learn Pipeline exported to a versioned champion pickle.
5. CI/CD	GitHub Actions runs Ruff, tests, fresh CI training and artifact upload.
6. Containerization	FastAPI prediction service packaged with Docker and a pinned serving stack.
7. Deployment	Two-replica Minikube Deployment exposed by Service and NGINX Ingress.
8. Monitoring	API logs/metrics collected by Prometheus and visualized in six Grafana panels.
9. Reporting	Report, evidence paths, screenshots and live demo are finalized for submission.

Repository: https://github.com/2025ae05188/heart-disease-mlops